

AI-Powered System Design & Code Evolution Platform

Final Requirements Specification

Project Team

May 12, 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Intended Audience	3
1.4	Key Terminology	3
2	Core Idea	4
3	Key Features	4
3.1	Visual System Modeling	4
3.2	Code Sync Engine	5
3.3	AI Coding Agent	5
3.4	Tool Calling System	5
3.5	Git Integration	5
3.6	Visualization Layer	5
3.7	Versioned System Model	6
3.8	System Engineering Extension (Post-MVP)	6
4	System Architecture	7
4.1	High-Level Diagram	7
4.2	Component Responsibilities	7
4.3	Technology Stack	7
5	Node System Design (Fixed Types)	9
5.1	Why Fixed Types?	9
5.2	Core Node Types	9
5.2.1	Service Node	9
5.2.2	API Node	9
5.2.3	Database Node	9
5.2.4	Message Queue Node (Optional for MVP)	10
5.3	Edge Types	10

6	Data Models	11
6.1	Diagram JSON Schema (Locked Contract)	11
6.2	MongoDB Collections	12
6.2.1	projects	12
6.2.2	diagrams (current state)	12
6.2.3	diagram_versions (immutable snapshots)	12
6.2.4	agent_logs	13
7	Tool Calling System	14
7.1	Tool Registry	14
7.2	Core Tools (MVP)	14
7.3	Tool Execution Flow	14
7.4	Idempotency Example	15
8	AI Agent Design	16
8.1	Agent State Machine	16
8.2	Diff-Driven Planning	16
8.3	Rollback and Partial Failures	17
8.4	Conflict Detection	17
8.5	Explanation Mode	17
9	Git Integration	18
9.1	Programmatic Git Operations	18
9.2	Commit Strategy	18
9.3	Branch Lifecycle	18
9.4	Visual Diff	18
10	End-to-End Workflow	19
11	MVP Scope (Must-Have)	21
11.1	Mandatory Features	21
11.2	Out of MVP Scope	21
12	Non-Functional Requirements	22
12.1	Performance	22
12.2	Reliability	22
12.3	Security	22
12.4	Maintainability	22
13	Risks and Mitigations	23
14	Appendices	23
14.1	Appendix A: Example Agent Plan (JSON)	23
14.2	Appendix B: Frontend State Management	24
14.3	Appendix C: Glossary of Acronyms	24

1 Introduction

1.1 Purpose

This document defines the complete functional and technical requirements for the **AI-Powered System Design & Code Evolution Platform** (codename: *CodeEvo*). The platform enables developers to visually design distributed systems using SysML-inspired diagrams, then uses an intelligent agent to automatically generate, modify, and evolve production-ready code, while keeping a Git repository fully synchronized.

1.2 Scope

The system covers:

- Visual diagram editor (drag-and-drop with fixed node types)
- AI agent with tool-calling capabilities (plan → execute → verify)
- Code generation for Spring Boot and Go microservices
- Git integration (auto-commit, branching, diff visualization)
- Versioned system model (MongoDB snapshots linked to commits)
- Interactive graph visualization of system evolution

1.3 Intended Audience

Developers, architects, and technical stakeholders who want to accelerate backend system design and prototyping through a visual, AI-augmented workflow.

1.4 Key Terminology

Term	Definition
Diagram	JSON representation of nodes and edges describing a system architecture.
Node	A visual component representing a Service, API, Database, etc.
Edge	A connection between nodes (REST call, event, DB connection).
Agent	LLM-based orchestrator that interprets diagram changes and invokes tools.
Tool	A backend function (e.g., <code>create_service</code>) that performs file/Git operations.
Version Snapshot	Immutable stored copy of a diagram at a specific commit.

2 Core Idea

The platform combines four pillars:

1. **Visual System Modeling** (Figma-like diagram editor)
2. **Code Generation Engine** (maps diagram nodes to real modules)
3. **AI Coding Agent** (understands changes, invokes tools, maintains consistency)
4. **Git Synchronization** (every agent action is committed and traceable)

Think: “*SysML + Git + Agent = Visual Dev Intelligence System*”

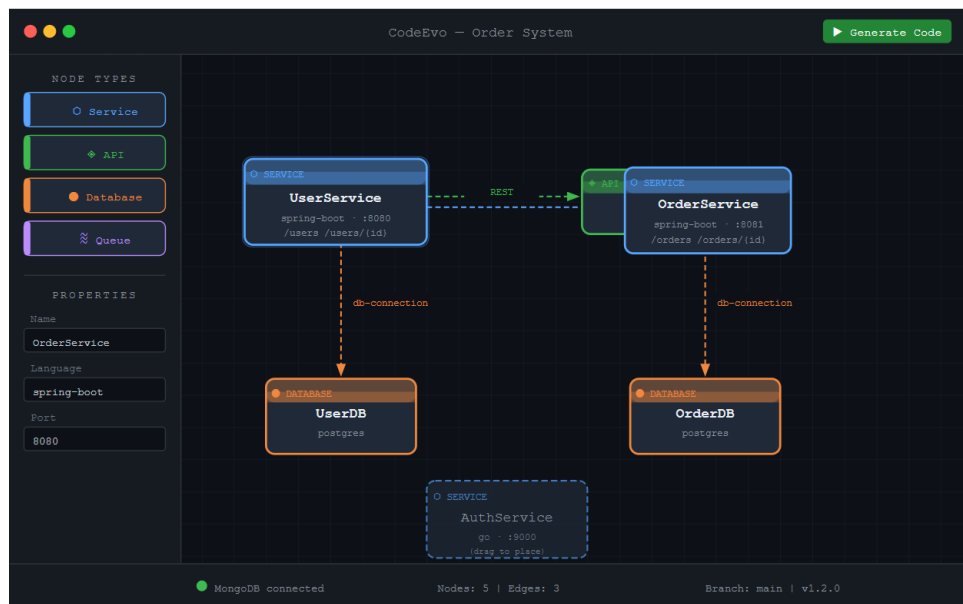


Figure 1: Mockup of the diagram editor showing services, API endpoints, and database nodes.

3 Key Features

3.1 Visual System Modeling

- Drag-and-drop canvas (React Flow) with predefined node types: Service, API, Database, Message Queue, External API.
- Edges representing REST calls, events, or database connections.
- Real-time persistence to MongoDB.
- Editing operations: rename, connect, delete, update properties.

3.2 Code Sync Engine

- Each diagram node maps to concrete code artifacts:
 - **Service** → folder + Spring Boot/Go project skeleton
 - **API** → controller method + route definition
 - **Database** → configuration + repository/model files
 - **Edge** → API client call or repository usage
- Bi-directional sync is **out of MVP scope** (diagram is source of truth).

3.3 AI Coding Agent

- Not a simple chatbot – it executes a deterministic loop: IDLE → PLANNING → EXECUTING → VERIFYING → COMMITTED.
- Receives a **diff object** between old and new diagram JSON, not the full diagram.
- Uses tool calling (e.g., `create_service`, `modify_api_contract`, `commit_changes`) with multi-step reasoning.
- Maintains consistency across services; supports rollback on failure.

3.4 Tool Calling System

- Tools are real backend functions (file ops, Git ops, code parsing).
- Tool registry with JSON schema validation.
- Idempotent design: repeated calls with same parameters do not cause duplication.
- Structured input/output for each tool.

3.5 Git Integration

- Auto-commit after every successful agent action.
- Each agent run creates a dedicated branch; merged only on full success.
- Visual diff: diagram changes ↔ code changes.
- Commit messages are generated by the agent describing the transformation.

3.6 Visualization Layer

- Interactive graph of services and dependencies.
- Time-based evolution view (how the system changed over commits).
- Click on node → view related code in the repo.
- Version-aware rendering (compare two snapshots).

3.7 Versioned System Model

- Every diagram save produces an immutable snapshot in MongoDB.
- Snapshot linked to Git commit hash.
- Diff logic between any two system states.
- Rollback capability (restore diagram and regenerate code).

3.8 System Engineering Extension (Post-MVP)

- Requirement → Design → Implementation traceability.
- Impact analysis: “If I change this service, what breaks?”
- Custom node templates (e.g., Auth Service template, Payment Service template).

4 System Architecture

4.1 High-Level Diagram

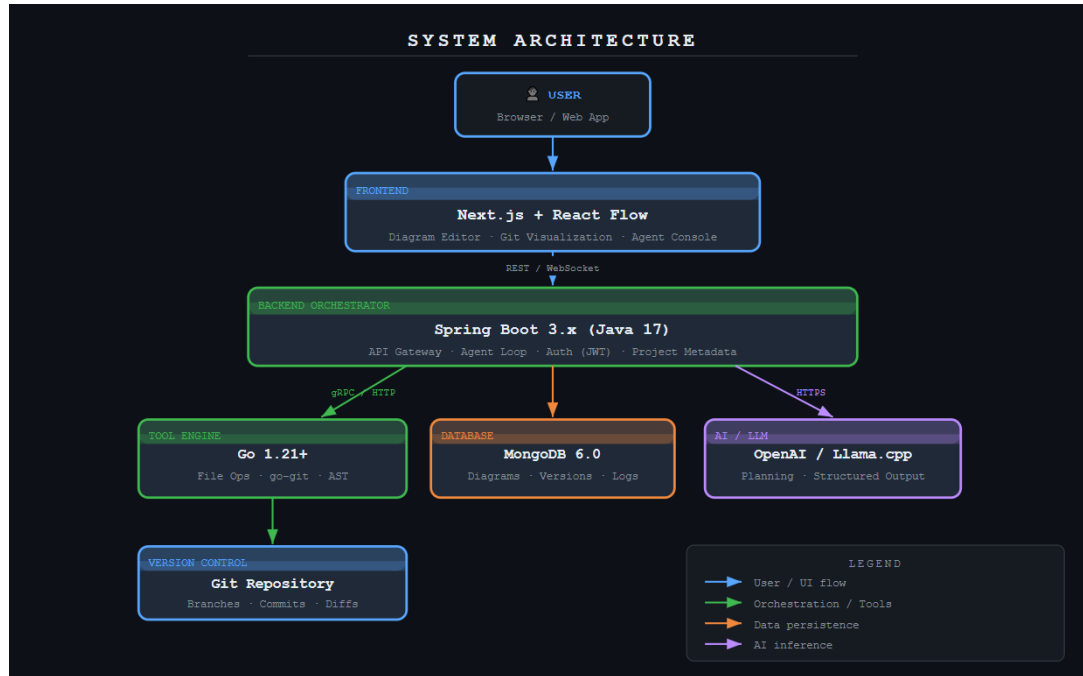


Figure 2: System components and their interactions.

4.2 Component Responsibilities

Component	Responsibility
Frontend (Next.js)	Diagram editor, Git visualization, agent interaction UI.
Backend Orchestrator (Spring Boot)	API gateway, agent loop management, authentication, project metadata.
Tool Execution Engine (Go)	File system operations, Git commands, code parsing/AST manipulation.
Database (MongoDB)	Diagram states, version snapshots, agent logs, tool execution history.
Git Repository	Source code versioning, branch management, commit history.

4.3 Technology Stack

- **Frontend:** Next.js, React Flow, Tailwind CSS, Axios
- **Backend (Orchestration):** Spring Boot 3.x, Java 17, Maven
- **Tool Engine:** Go 1.21+, go-git, standard library
- **Database:** MongoDB 6.0+, Mongoose (Node ODM) / Spring Data MongoDB

- **AI:** OpenAI API / local LLM (e.g., Llama.cpp) with structured output
- **Git:** go-git library (no CLI dependency)

5 Node System Design (Fixed Types)

5.1 Why Fixed Types?

Fully dynamic nodes lead to mapping ambiguity, agent confusion, and unreliable code generation. Therefore, we enforce a **Kubernetes-style resource model**: predefined types with flexible configuration.

5.2 Core Node Types

5.2.1 Service Node

```
1 {
2   "id": "uuid",
3   "type": "service",
4   "name": "UserService",
5   "config": {
6     "language": "spring-boot | go",
7     "port": 8080,
8     "endpoints": ["/users", "/users/{id}"],
9     "dependencies": ["OrderService", "AuthService"]
10  },
11  "position": { "x": 100, "y": 200 }
12 }
```

Maps to: microservice folder, controller/service/repo layers, build configuration.

5.2.2 API Node

```
1 {
2   "id": "uuid",
3   "type": "api",
4   "name": "GetUsers",
5   "config": {
6     "method": "GET",
7     "route": "/users",
8     "requestSchema": {},
9     "responseSchema": { "type": "array", "items": { "$ref": "User" } }
10  }
11 }
```

Maps to: controller method, route registration, OpenAPI annotation.

5.2.3 Database Node

```
1 {
2   "id": "uuid",
3   "type": "database",
4   "name": "UserDB",
```

```
5  "config": {
6    "engine": "postgres | mongodb",
7    "collections": ["users", "sessions"],
8    "connectionString": "${DB_URL}"
9  }
10 }
```

Maps to: application.properties, repository interfaces, entity/model classes.

5.2.4 Message Queue Node (Optional for MVP)

```
1  {
2    "id": "uuid",
3    "type": "queue",
4    "name": "OrderEvents",
5    "config": {
6      "provider": "rabbitmq | kafka",
7      "topics": ["order.created", "order.paid"]
8    }
9  }
```

5.3 Edge Types

```
1  {
2    "id": "uuid",
3    "from": "UserService",
4    "to": "OrderService",
5    "type": "rest | event | db-connection",
6    "config": {
7      "method": "POST",
8      "path": "/orders",
9      "eventType": "order.created"
10   }
11 }
```

6 Data Models

6.1 Diagram JSON Schema (Locked Contract)

This schema must be agreed upon in Week 1. Any change later breaks all layers.

```

1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "type": "object",
4   "required": ["version", "projectId", "nodes", "edges"],
5   "properties": {
6     "version": { "type": "string", "pattern": "^\\d+\\.\\d+\\.\\d+$" },
7     "projectId": { "type": "string", "format": "uuid" },
8     "nodes": {
9       "type": "array",
10      "items": {
11        "oneOf": [
12          { "$ref": "#/definitions/serviceNode" },
13          { "$ref": "#/definitions/apiNode" },
14          { "$ref": "#/definitions/databaseNode" },
15          { "$ref": "#/definitions/queueNode" }
16        ]
17      }
18    },
19    "edges": {
20      "type": "array",
21      "items": { "$ref": "#/definitions/edge" }
22    }
23  },
24  "definitions": {
25    "serviceNode": {
26      "type": "object",
27      "required": ["id", "type", "name", "config", "position"],
28      "properties": {
29        "id": { "type": "string", "format": "uuid" },
30        "type": { "enum": ["service"] },
31        "name": { "type": "string" },
32        "config": { "$ref": "#/definitions/serviceConfig" },
33        "position": { "$ref": "#/definitions/position" }
34      }
35    },
36    "serviceConfig": {
37      "type": "object",
38      "properties": {
39        "language": { "enum": ["spring-boot", "go"] },
40        "port": { "type": "integer" },
41        "endpoints": { "type": "array", "items": { "type": "string" } },
42        "dependencies": { "type": "array", "items": { "type": "string" } }
43      }
44    }
45  }
46 }

```

```

44     },
45     "position": {
46         "type": "object",
47         "properties": { "x": { "type": "number" }, "y": { "type": "number" } }
48     },
49     "edge": {
50         "type": "object",
51         "required": ["id", "from", "to", "type"],
52         "properties": {
53             "id": { "type": "string", "format": "uuid" },
54             "from": { "type": "string" },
55             "to": { "type": "string" },
56             "type": { "enum": ["rest", "event", "db-connection"] },
57             "config": { "type": "object" }
58         }
59     }
60 }
61 }

```

6.2 MongoDB Collections

6.2.1 projects

```

1 {
2     "_id": ObjectId,
3     "projectId": UUID,
4     "name": "E-Commerce System",
5     "ownerId": "user123",
6     "createdAt": ISODate,
7     "currentVersion": "1.2.0"
8 }

```

6.2.2 diagrams (current state)

```

1 {
2     "_id": ObjectId,
3     "projectId": UUID,
4     "currentVersion": "1.2.0",
5     "updatedAt": ISODate
6 }

```

6.2.3 diagram_versions (immutable snapshots)

```

1 {
2     "_id": ObjectId,
3     "projectId": UUID,
4     "version": "1.2.0",

```

```
5  "snapshot": { ... diagram JSON ... },
6  "gitCommitHash": "a1b2c3d4",
7  "createdAt": ISODate,
8  "agentRunId": UUID
9  }
```

6.2.4 agent_logs

```
1  {
2    "_id": ObjectId,
3    "runId": UUID,
4    "projectId": UUID,
5    "startTime": ISODate,
6    "endTime": ISODate,
7    "status": "success | failed | partial",
8    "plan": [ ... tool calls ... ],
9    "executionTrace": [ ... each tool input/output ... ],
10   "error": "optional"
11 }
```

7 Tool Calling System

7.1 Tool Registry

Each tool is registered with:

- Name (e.g., `create_service`)
- JSON schema for input parameters
- Output schema
- Go implementation function
- Idempotency key extraction logic

7.2 Core Tools (MVP)

1. `create_service` – Creates folder, boilerplate files, and build config for a service node.
2. `create_api` – Adds a controller method and route definition.
3. `connect_db` – Generates repository and configuration for database connection.
4. `update_file` – Modifies an existing source file (line-based or AST-aware).
5. `commit_changes` – Stages and commits all changes with a generated message.
6. `create_branch` – Creates a new Git branch.
7. `merge_branch` – Merges branch back to main (used after successful agent run).
8. `rollback` – Reverts to previous commit (used on failure).

7.3 Tool Execution Flow

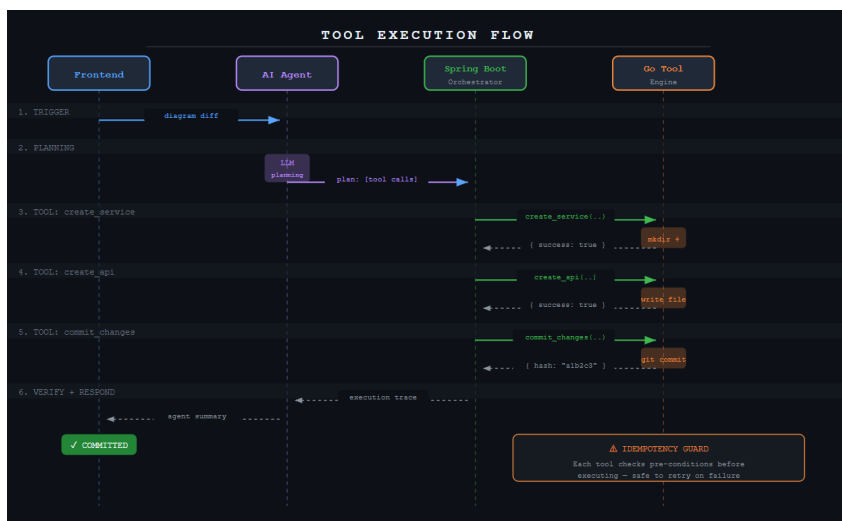


Figure 3: Sequence of tool invocation within an agent run.

7.4 Idempotency Example

```
1 func CreateService(input CreateServiceInput) error {  
2     path := filepath.Join(repoRoot, "services", input.Name)  
3     if _, err := os.Stat(path); !os.IsNotExist(err) {  
4         return nil // already exists, skip  
5     }  
6     // proceed with creation  
7 }
```

8 AI Agent Design

8.1 Agent State Machine

The agent follows a strict state machine to ensure consistency.

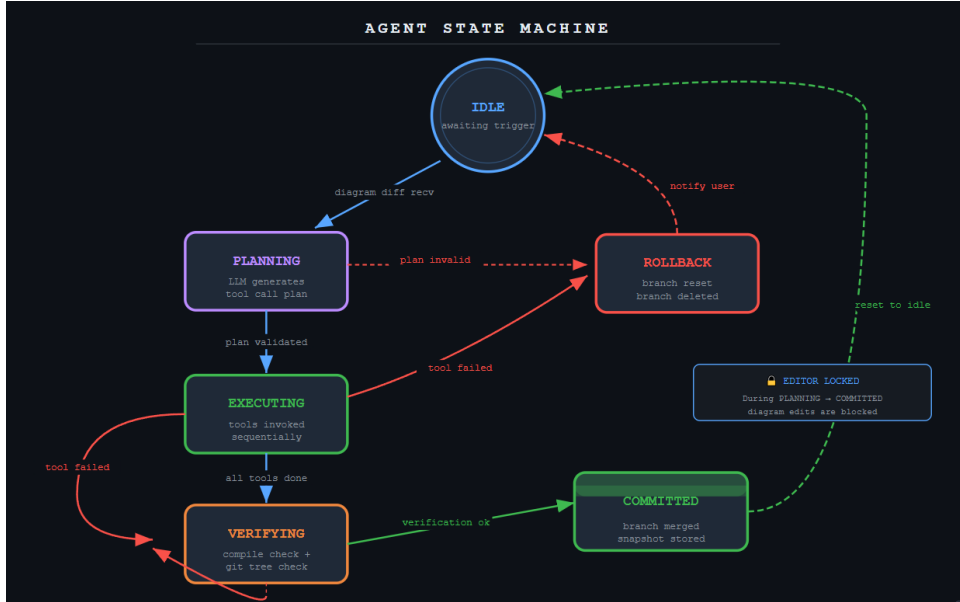


Figure 4: Agent state machine with transitions.

1. **IDLE**: Waiting for user request or diagram change.
2. **PLANNING**: Receives diff object; LLM generates a plan (sequence of tool calls with parameters). Plan is validated for syntax and feasibility.
3. **EXECUTING**: Executes each tool call in order. After each tool, updates execution trace.
4. **VERIFYING**: Checks that generated code compiles (optional) and that Git working tree matches expected state.
5. **COMMITTED**: Creates Git commit, merges branch (if successful), stores snapshot.
6. **ROLLBACK** (failure state): Reverts to last known good commit, deletes temporary branch.

8.2 Diff-Driven Planning

The agent never receives the full diagram JSON. Instead, a diff is computed:

```

1 {
2   "addedNodes": [ ... ],
3   "removedNodes": [ ... ],
4   "modifiedNodes": [ { "id": "...", "old": {...}, "new": {...} }
   ],

```

```
5  "addedEdges": [ ... ],  
6  "removedEdges": [ ... ]  
7  }
```

The LLM translates this diff into a plan. For example:

Diff: added Service "PaymentService"

Plan:

1. create_service("PaymentService", "spring-boot", 8081)
2. create_api("PaymentService", "POST", "/payments")
3. connect_db("PaymentService", "PaymentDB")
4. commit_changes("Add PaymentService with payments endpoint")

8.3 Rollback and Partial Failures

- Each agent run creates a new Git branch (e.g., `agent/run-<uuid>`).
- Tools are applied on that branch.
- If any tool fails, the entire run is rolled back by resetting the branch to the starting commit.
- Only after `VERIFYING` passes, the branch is merged into `main`.
- The diagram snapshot is stored only after a successful merge.

8.4 Conflict Detection

If the user edits the diagram while an agent run is in progress, the editor is locked with a visual indicator: *"Agent running – diagram locked"*. After the run finishes, the editor reloads the latest snapshot. This prevents corrupted intermediate states.

8.5 Explanation Mode

The agent outputs a human-readable reasoning summary alongside each plan. Displayed in the Agent Console panel.

9 Git Integration

9.1 Programmatic Git Operations

We use the `go-git` library for all Git interactions (no CLI). Operations:

- Initialize repo (if not exists)
- Create branch
- Add files
- Commit with custom message
- Merge branch
- Reset hard (for rollback)
- Diff between commits

9.2 Commit Strategy

- After every successful agent run → one commit on the temporary branch.
- Commit message format: `[Agent] <plan summary> (run-id: <uuid>)`
- Example: `[Agent] Add PaymentService with POST /payments endpoint (run-id: 123e4567)`

9.3 Branch Lifecycle

1. `main` is the stable branch.
2. Agent creates `agent/run-<uuid>` from `main`.
3. Applies tools, commits.
4. If verification passes: merges into `main`, deletes branch.
5. If fails: deletes branch without merging.

9.4 Visual Diff

The frontend can show:

- Diagram diff (side-by-side comparison of two snapshots)
- Git diff (file changes between two commits)
- Highlighting which diagram change caused which code change

10 End-to-End Workflow

The platform must support the following user scenario:

1. User opens web app (Next.js).
2. Creates a new project, names it “Order System”.
3. Drags a **Service** node onto canvas, names it **OrderService**.
4. Drags a **Database** node, names it **OrderDB**.
5. Connects them with an edge of type **db-connection**.
6. Clicks “**Generate / Update Code**”.
7. Agent receives diff (all new nodes/edges), plans:

```
create_service("OrderService", "spring-boot", 8080)
connect_db("OrderService", "OrderDB", "postgres")
commit_changes("Initial OrderService with DB connection")
```

8. Spring Boot orchestrator calls Go tool engine.
9. Go engine creates folder **services/OrderService/** with Maven structure, **application.properties** containing DB URL, and a repository class.
10. Git commit created on branch **agent/run-xxx**, then merged to **main**.
11. UI updates: shows system graph with new nodes, displays Git diff summary, and agent console shows reasoning.

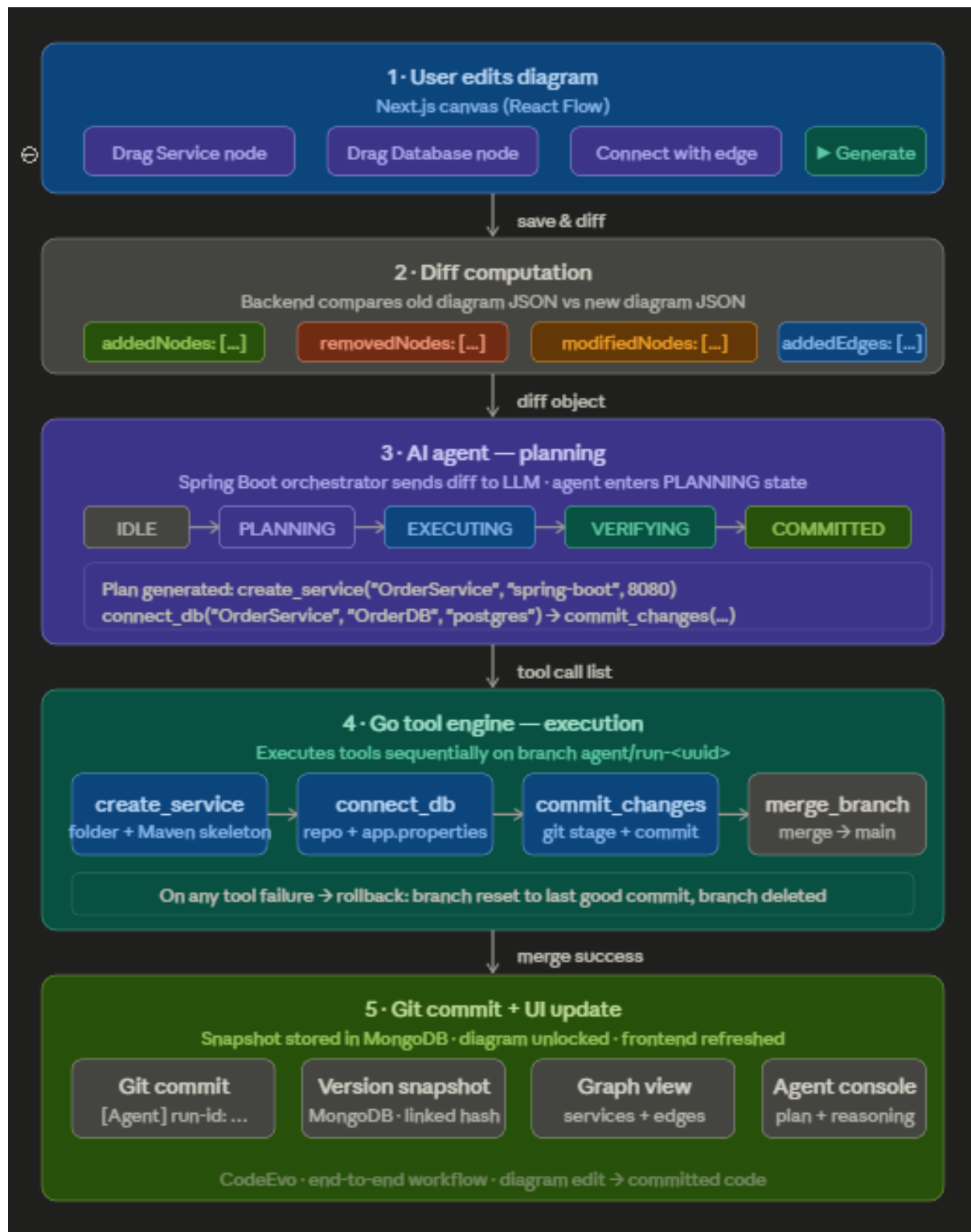


Figure 5: Complete end-to-end flow from diagram edit to committed code.

11 MVP Scope (Must-Have)

11.1 Mandatory Features

- Diagram editor with drag-and-drop for three node types: `Service`, `API`, `Database`.
- Persistence of diagrams in MongoDB.
- Code generation for:
 - One service (Spring Boot skeleton)
 - One API endpoint (controller method)
 - One database connection (repository + config)
- Agent with at least 3 tools: `create_service`, `create_api`, `commit_changes`.
- Git auto-commit after generation.
- Basic graph visualization of services.

11.2 Out of MVP Scope

- Full SysML support (sequence diagrams, state machines)
- Bi-directional sync (code $\rightarrow$ diagram)
- Complex AST parsing
- Full DevOps pipeline (CI/CD integration)
- Custom node templates

12 Non-Functional Requirements

12.1 Performance

- Diagram editor must support up to 100 nodes without noticeable lag.
- Agent planning should complete within 10 seconds (including LLM inference).
- Code generation for a single service must finish under 2 seconds.

12.2 Reliability

- Agent runs must be atomic: either all changes applied or none (branch rollback).
- MongoDB writes must be acknowledged before responding to user.
- Tool execution must be idempotent; retries safe.

12.3 Security

- User authentication via JWT (Spring Security).
- Projects isolated per user/team.
- Git credentials stored encrypted.

12.4 Maintainability

- Clear separation between orchestrator (Spring Boot) and tool engine (Go).
- Tool registry exposed via REST API with OpenAPI documentation.
- Diagram JSON schema versioned and validated on every save.

13 Risks and Mitigations

Risk	Impact	Mitigation
Agent produces invalid tool calls	High	Output schema enforcement; retry with fixed prompt
Partial execution leaves repo inconsistent	High	Branch-per-run + full rollback on any failure
LLM latency too high	Medium	Use smaller, faster model for planning; cache common patterns
MongoDB schema changes after development	High	Lock schema early; use migration scripts
User edits diagram during agent run	Medium	Lock editor UI; show agent status

14 Appendices

14.1 Appendix A: Example Agent Plan (JSON)

```

1 {
2   "runId": "550e8400-e29b-41d4-a716-446655440000",
3   "plan": [
4     {
5       "tool": "create_service",
6       "params": { "name": "PaymentService", "language": "spring-
7         boot", "port": 8081 }
8     },
9     {
10      "tool": "create_api",
11      "params": { "serviceName": "PaymentService", "method": "POST
12        ", "route": "/payments" }
13    },
14    {
15      "tool": "commit_changes",
16      "params": { "message": "Add PaymentService with POST /
17        payments" }
18    }
19  ]
20 }

```

14.2 Appendix B: Frontend State Management

Recommend using **Zustand** or **Redux Toolkit** for diagram state, with WebSocket updates for real-time collaboration (optional for MVP)

14.3 Appendix C: Glossary of Acronyms

Acronym	Meaning
AST	Abstract Syntax Tree
CRUD	Create, Read, Update, Delete
LLM	Large Language Model
MVP	Minimum Viable Product
REST	Representational State Transfer
SysML	Systems Modeling Language
UML	Unified Modeling Language

Conclusion

This requirements document defines a complete, executable specification for the AI-Powered System Design & Code Evolution Platform. By following the fixed node types, diff-driven agent, and branch-based rollback strategy. The platform will provide a novel, visual, and AI-augmented way to build backend systems with full Git traceability.